

PLAN DE BRANCHING - PROYECTO S.I.G.I.

Proyecto: Municipalidad Valle del Sol

1. Para qué es este documento

Acá dejamos por escrito la estrategia de ramas que usamos los tres para no pisarnos el código,

Refleja lo que ya hicimos en GitHub y lo que acordamos hasta el cierre del semestre.

2. Modelo elegido: GitHub Flow adaptado

No usamos Git Flow completo (sin develop ni release/* formales) porque el equipo es pequeño y

Usamos una variante de GitHub Flow:

Rama | Propósito

Por qué no Git Flow estricto: ramas develop y hotfix agregan overhead que en práctica no

gitGraph

3. Historial real del repositorio (referencia)

Ramas remotas observadas en GitHub:

Rama | Autoría aproximada | Contenido integrado

main | E

Commits de integración relevantes en main:

- 6ad90d3 - Merge PR #1 (Emilio_Dev)

- 3ad840

4. Convenciones de nombres

4.1 Ramas de funcionalidad

Formato recomendado:

<tipo>/<descripcion-corta-en-kebab-case>

Prefijo | Uso | Ejemplo

feature/ |

Alternativa usada al inicio: rama con nombre del integrante (Emilio_Dev, rodri). Funcionó al

principio,

4.2 Mensajes de commit

Preferimos mensajes en español, imperativo o descriptivo:

- Creacion API Gateway y filtro JWT

- Agrega

Evitamos commits genéricos tipo fix, cambios, asdf en entregas formales.

5. Flujo de trabajo del equipo

5.1 Diagrama del proceso

flowchart LR

A[Crear

5.2 Pasos detallados

1. Actualizar main local

git che

2. Crear rama de trabajo

git che

3. Commits frecuentes en la rama (unidades lógicas: un endpoint, una página, un fix).

4. Push

git push

5. Pull Request en GitHub hacia main

- Título

6. Merge tras aprobación (preferimos merge commit o squash según el PR; en PR #1-#3 usamos

merge es

7. Post-merge: quien mergeó avisa al grupo; todos hacen git pull en main.

6. Reglas del equipo (acuerdos)

Regla | Razón

No hacer

7. Asignación por integrante (referencia)

Integrante | Ramas / áreas típicas | Responsabilidad en PR

Hawk Du

En la práctica, los tres revisamos todo lo que podemos; esta tabla es la preferencia, no una

exclusión

8. Manejo de conflictos

1. En la rama feature:

git fetch

Si el conflicto es en package-lock.json o pom.xml, acordamos regenerar dependencias (npm

install / m

9. Ramas planificadas para cierre del proyecto

Rama propuesta | Objetivo | Responsable sugerido

feature/s

Todas fusionan a main vía PR cuando Docker y tests pasen.

10. Tags y releases (opcional)

Para la entrega final podemos etiquetar:

git tag -a v1.0-entrega -m "Entrega semestral SIGI - Mayo 2026"

git push o

Eso facilita al profesor clonar la versión exacta de la defensa sin depender del último commit

suelto.

11. Herramientas

Herramienta | Uso

GitHub |

12. Conclusión

Nuestra estrategia es simple y alineada con equipos pequeños: main protegida, trabajo en ramas

de featur

Para la entrega documental, este plan debe leerse junto con el Informe técnico

(INFORM

Elaborado por Hawk Durant, Emilio Jaramillo y Rodrigo Candia - Mayo 2026.